

AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

RISHI KHARYA, Indian Institute of Technology Bhilai, India

Rural water security depends heavily on small rainwater-harvesting ponds, yet choosing where to build one is usually done by eye and gives no quantitative basis for sizing. This project delivers a working web application that turns that judgement into an automated, explainable computation. A user selects any land area on an interactive map; the backend fetches a real Digital Elevation Model for exactly that area from OpenTopography, reconstructs the terrain, and routes surface water across it using depression filling, D8 flow direction and flow accumulation. It then ranks candidate pond sites using topographic wetness and position indices while explicitly excluding steep ground and the existing drainage network, delineates the catchment that drains to each site, and measures areas in a projected coordinate system. Historical rainfall from the Open-Meteo archive is combined with the catchment and the basin's own geometry to estimate the water volume that can realistically be collected and the pond depth required to hold a year's runoff. Every result — suggested pond, catchment, contours and existing drainage — is overlaid on the map, and can be exported as an image, GeoJSON or JSON. The system is deployed as a single lightweight process on the provided memory-constrained servers, completes an analysis in about twelve seconds (instant when cached), and is covered by 71 automated tests. Nothing is hard-coded to any one location.

Additional Key Words and Phrases: Village Pond Planning, Geospatial Analysis, Rainwater Harvesting, Catchment Estimation, Web GIS, REST APIs

ACM Reference Format:

Rishi Kharya. 2026. AI-based Village Pond Planning System: CSD Assignment 1 – Final Technical Report. 1, 1 (September 2026), 9 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Access to water in rural areas is strongly tied to local rainwater harvesting. Small village ponds (tanks) capture monsoon runoff, recharge groundwater and provide water through dry months. But their effectiveness depends on *where* they are built: a pond must sit where the land naturally funnels runoff toward it and where the ground forms a basin that holds water, rather than on a slope or inside a stream. Making that judgement well requires reading terrain, estimating how much land drains to a point, and knowing how much rain falls there.

The goal of this assignment is an AI/geospatial-assisted web application that performs this reasoning automatically. A user selects a region on a map and the system recommends suitable pond locations, the catchment feeding each, and the water volume each could collect — all visualised on the map. The rest of this report covers the requirements (Section 2), the architecture (Section 3), the algorithms (Section 4), the implementation (Section 5), the mapping to core system design themes (Section 6), results and performance (Section 7), and a discussion of limitations (Section 8).

Author's address: Rishi Kharya, Indian Institute of Technology Bhilai, Bhilai, India, kharyarishi1@gmail.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

1.1 Motivation

Manual site selection is slow and inconsistent. A surveyor must walk the land and infer drainage by eye; there is no centralised, easy way to combine terrain shape with historical rainfall; and the result is qualitative, offering no basis for deciding pond depth or storage capacity. An automated tool reduces a large search space to a small, justified shortlist of sites — each with quantitative catchment and volume figures — that a village administrator can then inspect on the ground. It also makes the reasoning transparent and repeatable for any location, not just areas an expert already knows.

1.2 Scope of the Project

The system estimates, for a user-selected area: suitable pond locations, the catchment area draining to each, the pond's own footprint, the expected collectable water volume, and the depth required to hold a year's runoff, all overlaid on an interactive map and exportable. It does *not* perform the civil/structural engineering design of the pond (embankment, spillway, soil mechanics), it does not detect ponds that already physically exist from satellite imagery, and it does not model sub-surface hydrogeology. Results are decision support at survey-shortlist accuracy, bounded by the resolution of freely available elevation and rainfall data.

2 PROBLEM STATEMENT AND REQUIREMENTS

The system must let a user select a land area on a map and, for that area, generate and visualise the suggested pond location, the catchment area, and the expected collectable water volume. Table 1 restates the functional requirements and maps each to the module that implements it.

Table 1. Functional requirements and where they are implemented

Requirement	Implemented in
Interactive map + satellite imagery	frontend/MapView (Leaflet; OSM base + Esri toggle)
Select land area on the map	frontend/App, MapView (view-select / two-click box)
Terrain / elevation acquisition	elevation_api.py (Open-Topography)
Contour visualization	contour_lines.py + MapView
Suitable pond-site identification	pond_locator.py
Catchment area estimation	catchment.py + geometry_utils.py
Historical rainfall query	rainfall_api.py (Open-Meteo)
Runoff / water-volume estimation	water_volume.py
Pond depth / storage recommendation	water_volume.py (storage curve)
Combined overlay / results view	frontend/MapView, ResultsPanel

2.1 Non-Functional Requirements

Performance: a full analysis (including the live DEM fetch) should complete in well under a minute; repeat analyses of the same area should be near-instant. **Resource use:** the service must run within the provided containers' hard limit of 512 MB RAM and small storage. **Availability:** the server runs detached and stays up independently of any login session. **Resilience:** external-API failures must degrade gracefully (documented fallbacks, clear error messages) rather than crashing an analysis. **Usability:** the tool must work in a modern desktop browser with no installation and no configuration. **Portability:** the same build must run unchanged on any of the four provided systems.

3 SYSTEM ARCHITECTURE AND HIGH-LEVEL DESIGN

The system is a client plus a single backend service that also serves the client. The React front-end runs in the browser; it calls one REST endpoint on the FastAPI backend, which orchestrates two external data APIs (elevation and rainfall) and the in-process analysis pipeline, then returns a combined JSON payload that the front-end renders as map overlays. There is no separate database tier (Section 5.3). Figure 1 shows the data flow.

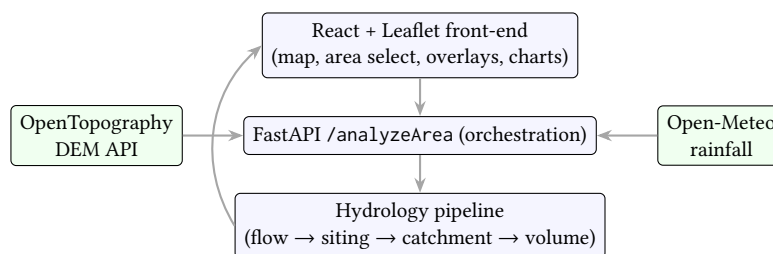


Fig. 1. High-level system architecture and data flow.

Each analysis module has one non-overlapping responsibility: `elevation_api` (fetch/parse the DEM), `terrain_grid` (grid and slope), `catchment` (depression filling, flow direction, flow accumulation, watershed delineation), `pond_locator` (site scoring and ranking), `geometry_utils` (metric area/perimeter), `rainfall_api` (rainfall), `water_volume` (runoff, capacity, storage curve), `contour_lines` (contours for display) and `visualization` (PNG render). The route only wires them together.

3.1 Technology Stack

Backend: Python with FastAPI (async REST + auto-generated OpenAPI docs); NumPy and SciPy for the raster/hydrology computations; Shapely and pyproj for geometry and UTM projection; contourpy for contour extraction; httpx for async external calls; matplotlib (lazily loaded) for server-side figure rendering. **Front-end:** React (built with Vite) and Leaflet, with OpenStreetMap as the base map and Esri World Imagery as a satellite toggle. **External APIs:** OpenTopography Global DEM (SRTMGL1, 30 m) for elevation, Open-Meteo historical archive for rainfall, and OpenStreetMap Nominatim for place search. **Deployment:** the compiled front-end is served as static files by the same FastAPI process, run detached inside the provided Docker containers.

Deviations from the brief's suggested stack are deliberate. The brief suggested per-point elevation services and a PostGIS database; we instead use OpenTopography, which returns a whole DEM raster for the selected block in a single request (avoiding per-coordinate rate limits that made point APIs unusable in testing), and we use no database because

the analysis is stateless and the containers are storage-constrained (Section 5.3). The satellite-based existing-pond detection (OpenCV) from the high-level design was scoped out of this phase.

4 METHODOLOGY

This section describes the actual algorithms and data pipeline.

4.1 Terrain and Elevation Analysis

For the selected bounding box the backend requests a DEM from the OpenTopography Global DEM API (SRTMGL1, 30 m) in ESRI ASCII-grid (AAIGrid) format, so it can be parsed with NumPy and needs no native GDAL/rasterio libraries. The raster is oriented so latitude increases with row index, no-data cells are in-filled, and if the raster exceeds a cell cap it is stride-downsampled to stay within memory. Local slope is computed with the Horn (1981) finite-difference gradient, $\text{Slope} = \arctan\sqrt{(dz/dx)^2 + (dz/dy)^2}$, with cell spacing converted from degrees to metres. Elevation contour lines for display are extracted from the DEM with contourpy.

4.2 Catchment Area Delineation

Artificial pits from the coarse DEM are first removed with the Priority-Flood algorithm (Barnes et al., 2014), an $O(n \log n)$ depression fill. D8 flow direction (O’Callaghan & Mark, 1984) then routes each cell to its steepest downhill neighbour, and flow accumulation — the count of upstream contributing cells — is computed by processing the flow graph in topological order. The catchment of a candidate site is obtained by tracing this flow network *backwards* from the site: a neighbour is added whenever its flow direction points at the current cell, until no upstream contributors remain. The resulting cell mask is unioned into an exact polygon (not an approximating hull, which was found to overestimate area) and projected into the local UTM zone — chosen automatically from the block’s location — so area and perimeter are measured in metres. *Assumptions/limitations:* D8 assigns all flow to one neighbour, and at 30 m resolution small valley-scale drainage on flat terrain is only coarsely resolved, so catchments there are conservative.

4.3 Rainfall Data Integration

Mean annual rainfall for the block’s centre is obtained from the Open-Meteo historical archive (daily precipitation_sum), averaged over the last several complete years to smooth wet/dry-year variation; a monthly climatology is also returned for display. Every call has a timeout and, if the archive is unreachable, the system falls back to a documented default annual rainfall and marks the source accordingly, so a volume estimate is always produced.

4.4 Pond Siting and Water Volume

Sites are scored by combining the Topographic Wetness Index, $\text{TWI} = \ln(a/\tan \beta)$, and the Topographic Position Index, $\text{TPI} = z - \bar{z}_{\text{nbhd}}$, as $\text{Score} = 0.5 \text{TWI}_n + 0.5 (-\text{TPI})_n$, then excluding cells steeper than 15° , cells in the existing channel network (flow accumulation above the 97th percentile plus a buffer), and candidates whose flood-fill footprint cannot be bounded. The annual runoff a catchment delivers is estimated by the Rational method $V_{\text{runoff}} = C P A_{\text{catchment}}$ ($C = 0.3$ by default), the basin storage capacity is measured directly from the DEM as $\sum (w - z_i) A_{\text{cell}}$, and the collectable volume is $\min(V_{\text{runoff}}, V_{\text{capacity}})$. A stage–area–volume curve and the depth needed to hold one year’s runoff are also computed.

5 IMPLEMENTATION

5.1 Backend and API Design

The backend exposes a small REST surface (Table 2). Requests and responses are JSON; the schema is defined with Pydantic models and interactive documentation is auto-generated at /docs. There is no authentication — it is an open, read-only analysis tool for the demo. Errors from third-party APIs are handled explicitly: a malformed or out-of-range area returns 422, an unreachable elevation service returns 502 with a clear message, rainfall failures fall back silently to a default, and any unexpected stage failure returns 500 naming the stage.

Table 2. REST API endpoints

Method & path	Purpose
POST /analyzeArea	Analyse a selected map region (bbox)
POST /analyzeArea?format=image	Same, returns a rendered PNG
POST /analyzeContour	Analyse an uploaded contour KML/KMZ (Phase 2)
GET /health	Service health check
GET /docs	Auto-generated API documentation

5.2 Frontend and Visualization

The front-end (Figure 2) is a single-page React + Leaflet app. The user searches a place by name (which flies the map there), frames a village-sized block, and selects it either with a one-click “use current view” or by clicking two opposite corners of a rectangle. On analysis, the map overlays the DEM contours, the existing drainage network (blue), and for each ranked site the pond footprint (a filled region labelled with its area), its catchment (dashed outline) and a location marker. A side panel lists, per site, the catchment area, pond footprint, annual runoff, basin capacity, collectable volume, required depth and a storage-curve chart, plus a monthly rainfall chart. Results export as PNG, GeoJSON or JSON.

5.3 Database and Storage

The system uses *no persistent database*. Each analysis is computed on demand from live external data and returned directly; there is no cross-session state to store. To avoid repeated external calls, fetched DEMs (keyed by bounding box) and rainfall results (keyed by centroid) are held in an in-memory cache, so re-analysing the same or an adjacent area is instant. This choice is deliberate: the provided containers are storage- and memory-limited, a database tier would add weight and a failure mode for no functional benefit here, and statelessness keeps the same build trivially portable across the four systems. Exported GeoJSON/JSON files let the user persist any result locally.

6 CSD THEMES AND TOPICS APPLIED IN THE PROJECT

Table 3 maps core system-design themes to where they appear in this project. Themes that were deliberately not implemented are marked as such, with the reason.

The three most significant decisions were caching, resilience and the monolith choice. **Caching** matters because a single analysis is dominated by the ~12 s external DEM download; caching by bounding box turns any repeat or nearby

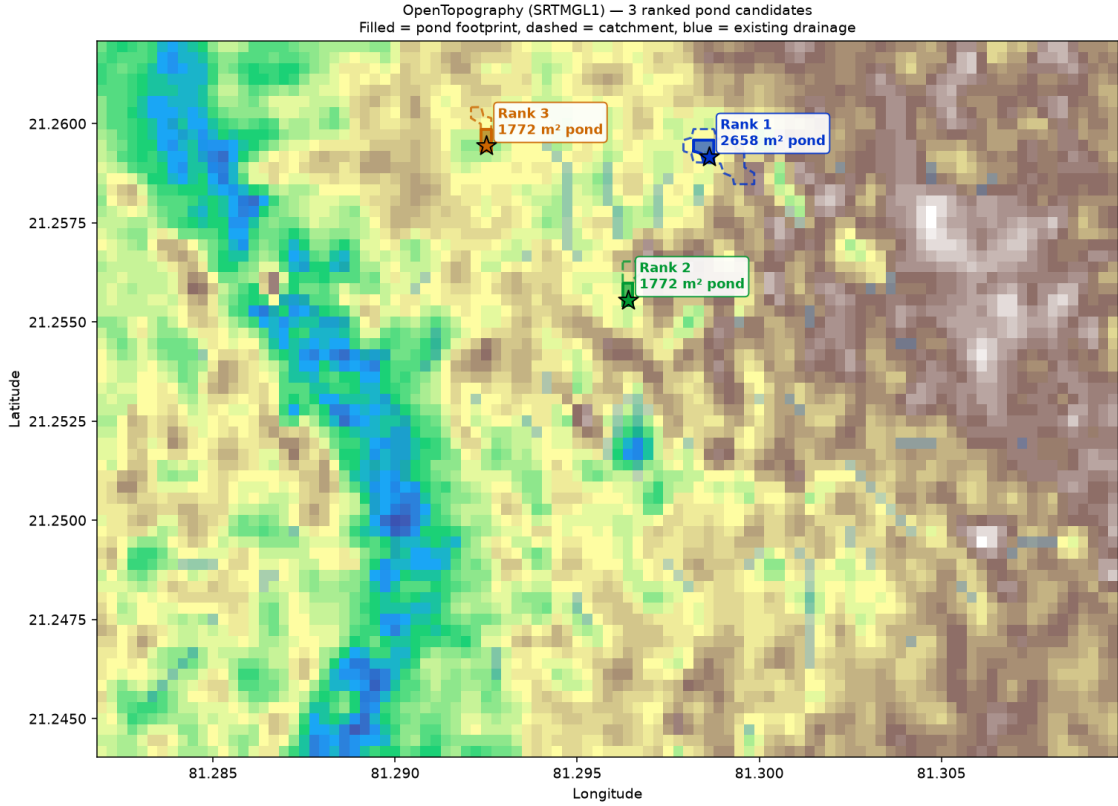


Fig. 2. Application results for the sample region: DEM terrain shading, the existing drainage network (blue), and three ranked pond sites (filled footprints) with their catchments (dashed), all kept off the watercourses.

selection into an instant response, which is what makes iterative exploration usable. **Resilience** was forced by reality: during development the free per-coordinate elevation service hit hard rate limits and other providers were unreachable, so we switched to OpenTopography’s single-request raster and made every external call fail gracefully (rainfall falls back to a default, elevation returns an actionable message). **The monolith** was chosen over the microservice design in the original HLD specifically because the deployment targets are 512 MB containers — one dependency-light process that serves both the API and the web app is the design that actually fits the hardware.

7 RESULTS AND EVALUATION

Figure 2 shows the system on the provided sample region (near Raipur, Chhattisgarh). The 30 m DEM (elevation 266–298 m) reproduces the known landscape: the diagonal blue band is the main stream valley with tributary channels, and all three suggested ponds are correctly placed on drainable ground away from that network. Table 4 gives the numbers.

The quantities are internally consistent: the catchment (the water source) is always at least as large as the pond footprint (where water collects), and the collectable volume is the smaller of what the catchment delivers and what

Table 3. Mapping of core CSD themes/topics to their use in this project

CSD Theme/Topic	Where used in the project	Justification / design rationale
API Design (REST)	POST /analyzeArea, /analyzeContour, /health	Clear JSON request/response contract via Pydantic; auto-generated OpenAPI docs; a query flag (format=image) switches representation without a new route.
Caching	In-memory cache of DEM (by bbox) and rainfall (by centroid)	The external DEM fetch dominates latency (~12 s); caching makes repeat/adjacent analyses instant and removes duplicate external calls.
Concurrency / Asynchronous Processing	async route + httpx.AsyncClient for the DEM fetch	The request is I/O-bound on the external API; async keeps the event loop free and the (fast) NumPy pipeline runs after the await.
Monolith vs. Microservices	Single FastAPI process serving both API and static front-end	Chosen over microservices because the containers are tiny (512 MB) and one process minimises memory, deployment steps and failure modes.
Error Handling and Resilience	Rainfall fallback, DEM 502 with guidance, bbox validation, retry/backoff	The system depends on free external APIs, so it must degrade gracefully rather than fail an analysis; each failure has a defined behaviour.
Algorithms and Complexity	Priority-Flood ($O(n \log n)$), D8 + flow accumulation ($O(n)$), BFS watershed	Optimal/linear algorithms keep the compute negligible (~0.1 s) so total time is dominated by the network, not the CPU.
Geospatial / Projection	UTM projection (pyproj), exact cell-union polygons (Shapely)	Areas must be measured in metres, not degrees; the exact union avoids the ~78% overestimate a convex hull produced.
Resource-Constrained Design	Lazy matplotlib import, grid cap + down-sampling, point caps	Keeps resident memory (~160 MB) well under the 512 MB container limit.
Testing Strategy	71 unit/integration tests (Section 7)	Each module is tested in isolation plus end-to-end, giving confidence the pipeline is correct and refactors are safe.
Containerization / Deployment	Runs in the provided Docker containers; scripted pull + restart on internal port 5000 (forwarded to 5228)	Reproducible, one-command redeploy; uses the pre-installed environment rather than a duplicate venv to save storage.
Version Control	Git + GitHub, incremental commits	History documents each change and bug fix and keeps the deployed servers in sync via git pull.
Load Balancing	Not implemented	A single instance meets the expected demo load within the container; an Nginx reverse proxy across instances would be the scale-out path.
Authentication / Authorization	Not implemented	The tool is an open, read-only analysis service for the demo; no user data is stored.
Database Indexing	Not applicable	No persistent database (Section 5.3); state is transient and cached in memory.
CI/CD	Not implemented	Deployment is a scripted git pull + restart; tests are run manually before deploy.

the basin can hold. Cross-checking against Phase 2 (which used a 1 m contour map of the same area) confirmed the pipeline places ponds in the same drainable zones and away from the river.

Table 4. Ranked pond candidates for the sample region ($C = 0.3$). Catchment area is always $\geq$ pond footprint, as expected.

Rank	Location (lat, lon)	Catch. (m ²)	Pond (m ²)	Collect. (m ³ /yr)	Depth (m)
1	21.2592, 81.2986	6 202	2 658	1 901	2.5
2	21.2556, 81.2964	2 658	1 772	877	1.0
3	21.2594, 81.2925	2 658	1 772	877	1.0

7.1 Performance

A full analysis of a fresh area completes in about 12 s, of which roughly 11.6 s is the external OpenTopography DEM download and ~ 0.1 – 1.5 s is the entire CPU pipeline (flow routing, siting, catchment, geometry, volume). Cached areas return in well under a second. Resident memory is about 160 MB, comfortably below the 512 MB container limit. The backend passes 71 automated tests spanning the parser, terrain grid, flow/catchment, pond locator, geometry, elevation parsing, rainfall, water volume and the image-rendering path.

8 DISCUSSION AND LIMITATIONS

The core pipeline works reliably and its behaviour is explainable end to end, and switching to OpenTopography’s single-request raster made the system dependable where per-coordinate elevation APIs had failed. The main limitations come from free-data resolution and scope. The 30 m DEM under-resolves small valley-scale drainage on flat terrain, so catchments and ponds there are modest and the D8 watershed can fragment; areas with real relief yield much larger, clearer catchments. The rainfall archive is occasionally unreachable, in which case a documented fallback value is used and flagged, making those volume figures indicative rather than exact. The system also reasons purely from terrain and rainfall: it does not know whether water already physically exists at a site, and it does not perform the civil design of the pond. Finally, integer-metre vertical precision in the DEM makes very gentle slopes read as near-zero. These are properties of the input data and the chosen scope, and are stated so the recommendations are used as a survey shortlist rather than a final design.

9 AI TOOL USAGE DECLARATION

Claude (Anthropic) was used as a coding and debugging assistant throughout development: to draft initial implementations of individual modules, to explain the underlying terrain-analysis algorithms, to help diagnose and fix real issues found during testing (for example, correcting a case where the pond footprint could exceed its catchment, resolved by clipping the footprint to the catchment; diagnosing external-API rate limits and adding graceful fallbacks; and cleaning up the drainage-network visualisation), and to help reformat content into this report template. All AI-assisted code and text were reviewed, understood and modified by the author before submission, and the algorithms and design trade-offs described in Section 4 can be explained in full.

A SOURCE CODE AND REPOSITORY

The complete source code, commit history and this report are at <https://github.com/krish1809/village-pond>. The live application is at <http://10.1.75.79:5228/> (API docs at /docs). The repository is organised as: backend/ — the FastAPI service, with modules/ (the analysis modules), api/routes/ (the endpoints), models/ (Pydantic schemas) and tests/;

frontend/ — the React + Leaflet application (src/); scripts/ — the build and deploy scripts; docs/ — design documents and this report; and the sample contour file at the project root.